

Can This Agent Even Do That?

Pre-Execution Validation for Agent Skills

Introduction

Organizations adopting tool-using agents need automation that completes requested work, not just plausible plans. Our business objective is to reduce wasted inference and prevent workflows that cannot deliver their promised outcome. An authorized agent may still lack an essential capability, input, or valid coordination path; retries consume resources without removing that blocker. This exposes a safety gap between permission to execute and ability to achieve the goal: authorization does not establish reachability, and checking results afterward cannot prevent a doomed attempt. Aligned with the conference's focus on Agents and multi-agent safety, we contribute SkillC, an implemented Python compiler that combines capability, protocol, and symbolic goal checks into an evidence-bearing pre-execution decision, retaining independent authorization and runtime verification. Our experiments show 90.47% lower model-token use for repeated impossible-task attempts with simulated tools, and 46.74% lower use on real backends without losing achievable-task completions. We explain implementation, formal guarantees, trade-offs, and proposed Microsoft Agent Framework (MAF) integration. This is our first MLADS submission.

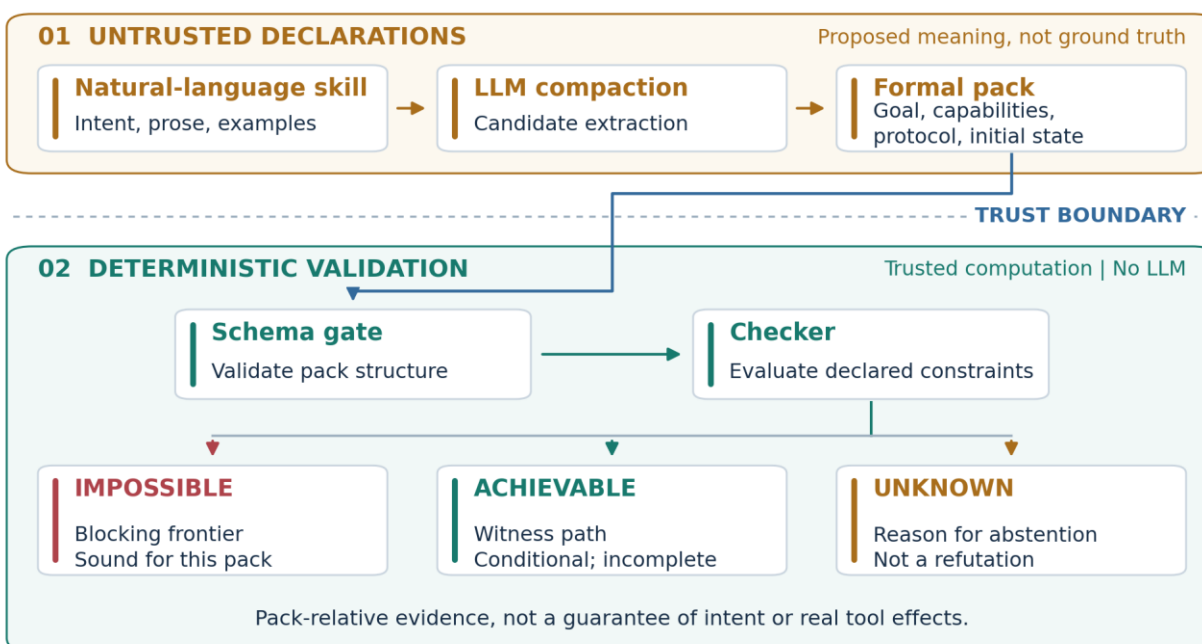


Figure 1. From an untrusted skill description to a deterministic, evidence-bearing verdict. Guarantees are relative to the declared pack; intent fidelity and actual tool effects remain separate obligations.

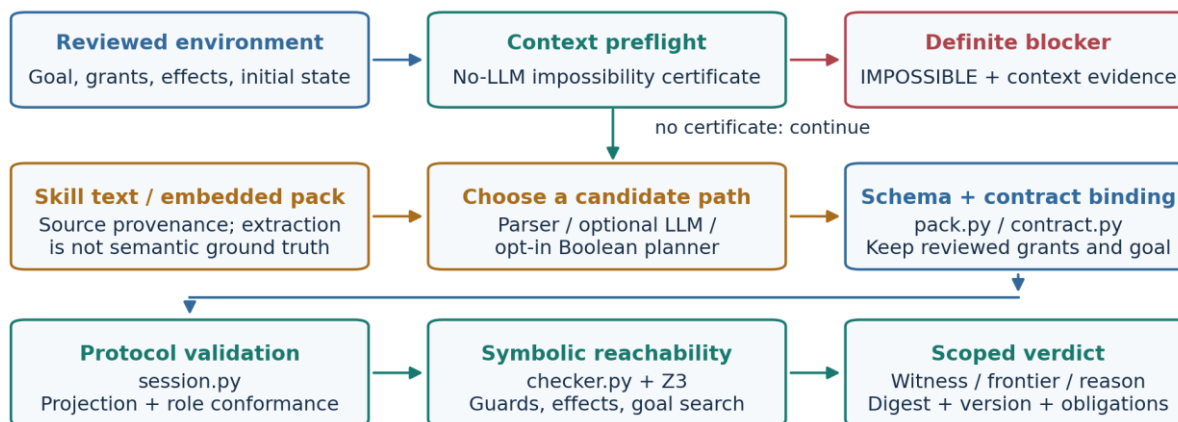
Related Work

SkillC combines STRIPS planning [1], multiparty session types and global-type analyses [2,4,5], and Z3 [3]. ADAS and AFlow generate agentic workflows [6,7]; multi-agent failure analysis motivates stronger coordination checks [8]. VeriGuard addresses policy safety [9], and SkillSpector screens skill bundles [10]. Our complementary question is static goal achievability under declared capabilities and protocols. We compare against the same agent without SkillC, not against these systems.

Methodology

Compile to an inspectable pack. Markdown, embedded packs, and optional Azure OpenAI/Anthropic compaction share one JSON schema. A separately reviewed contract fixes the goal, grants, guards/effects, and initial state. Extraction proposes a protocol; it cannot establish the truth of those declarations. A context-only preflight can reject impossible work before any compaction call.

IMPLEMENTED | SkillC compiler and admission components



ACHIEVABLE is conditional | IMPOSSIBLE has a scope | UNKNOWN is an abstention

Figure 2. Implemented SkillC components. The context shortcut may skip extraction; every candidate is validated. Extraction proposes structure; it does not determine the verdict.

Preserve decision scope. planning.py constructs checked alternatives for single-role Boolean contracts; unsupported synthesis or search limits abstain. session.py checks projection and role conformance; checker.py searches guarded world/protocol states. Malformed input is an error; unsupported or limited search is UNKNOWN, not approval. Protocol rejection blocks that workflow; goal-wide rejection requires an independent capability-context certificate. Verdicts retain a witness or blocking frontier, scope, digest, and deferred obligations.

Table 1. Three complementary static-validation signals.

Signal	Check	Evidence
Tools and effects	Capability reachability	Missing capability / witness action
Coordination	Protocol conformance	Mismatch / conformant protocol
Goal and refinements	Logical reachability	Blocked guard / satisfying model

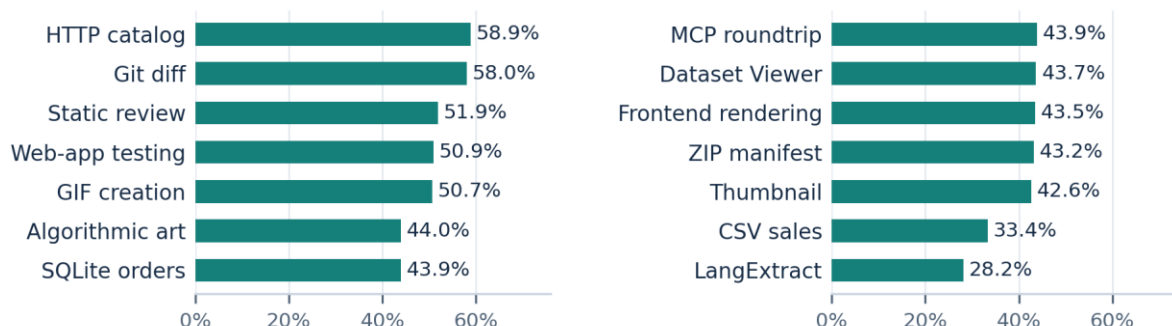
Data

The live study uses 13 Model Context Protocol (MCP) adapters plus LangExtract, each in four environments: complete, missing essential operation, missing optional diagnostics, or missing required input. Its 28 positive and 28 negative cases use fixed reviewed contracts, physically enforced restrictions, non-sensitive fixtures, and one trial per configuration using gpt-5.4-2026-03-05. Separately, five public skill profiles have five simulated-tool trials each; an independent oracle identifies four impossible profiles for reuse accounting. Metrics are provider tokens, independently verified completion, false rejection, and abstention: low token use alone is not success. Neither study estimates production prevalence.

Results

Token reduction by task family: deterministic gate vs ungated

Four environments per family; with and without SkillC, 2/2 achievable cases complete.



Observed single-pass comparisons, not confidence intervals or a production distribution.

Figure 3. Fourteen task families, four environments each. Reduction = $100 \times (\text{ungated tokens} - \text{gated tokens}) / \text{ungated tokens}$. The overall 46.74% uses summed token totals, not the mean of these percentages.

Live execution. The agent used 18,758 tokens with SkillC versus 35,220 without it, saving 16,462 (46.74%) while completing all 28 achievable cases in each configuration. LLM compaction used 84,334 tokens, 139.45% more than the ungated baseline. All three SkillC configurations rejected 28/28 impossible cases: TP=28, FP=0, FN=0, TN=28 (positive = IMPOSSIBLE), with zero false rejections or abstentions. The executor's 252 tokens are not an equal-success saving: runtime verification caught an empty LangExtract output, leaving 27/28 completed.

Across families. Deterministic gating reduces tokens in every family, with two of two achievable cases completed both with and without SkillC. Savings range from 28.2% for LangExtract to 58.9% for HTTP catalog. The bars show one pass per scenario, not confidence intervals.

Evidence accounting. The MCP portion records 260 initialized sessions, 331 tool invocations, and 227 Azure calls. Adding 24 corrected LangExtract calls gives 251 calls and 138,564 tokens across the four configurations. The separate 7,563-token diagnostic run is retained but excluded.

Table 2. Corrected repeated-use comparison: four independently impossible tasks.

Accounting basis	With SkillC	Without SkillC	Reduction
First use: one request per task	22,892	48,033.8	52.34%
Five requests per task (20 total)	22,892	240,169	90.47%

Why reuse matters. Compaction is paid once per unchanged skill; the first-use baseline sums five-trial means, while reuse counts all 20 measured attempts. These real model calls use simulated tool actions, not the live backends above. The fifth profile, web-app testing, is excluded: its goal was achievable despite protocol rejection. Counting prevented successes as savings would be misleading. The historical 85.28% measured-subset and 98.68% partly estimated comparisons are detailed separately in S12.

What the gate does not guarantee. Our original 15-case corpus exposes two planted intent/payload false admissions. Its matrix and separate six-case extension are in S8; capability/coordination examples are in S3.

Microsoft Agent Framework integration

Design, not a deployment claim. This Python adapter is not implemented or benchmarked here. Gate before inference or in the first workflow executor. Bind registered/discovered tools to reviewed semantic contracts; names and schemas cannot supply business guards or effects. An ACHIEVABLE verdict remains conditional, not authorization.

PROPOSED ONLY | Microsoft Agent Framework integration

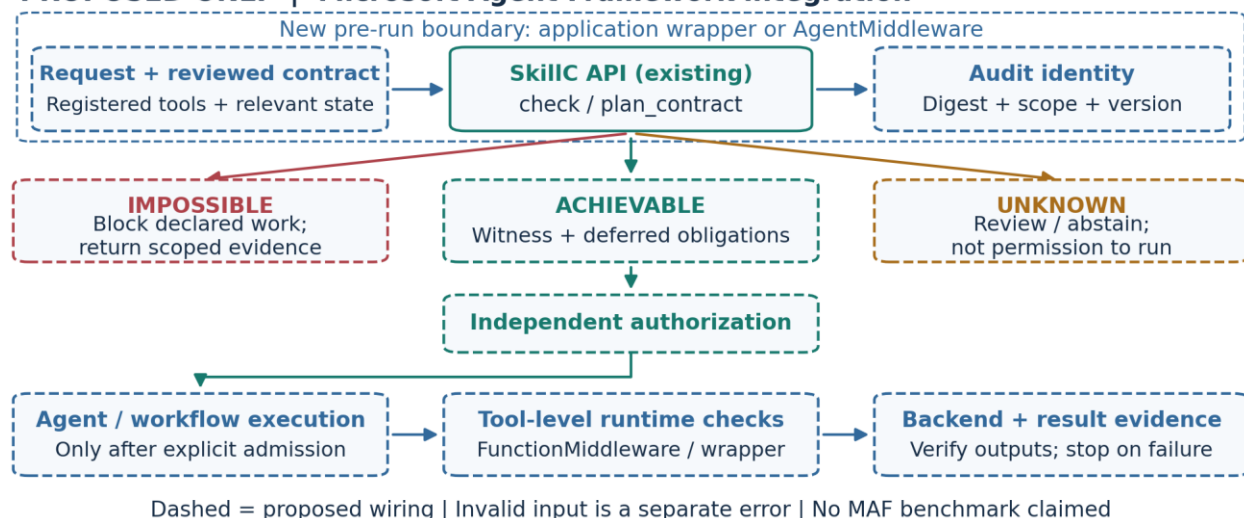


Figure 4. Proposed Microsoft Agent Framework adapter. Dashed boxes are new integration work, not measured features. Admission remains separate from authorization and runtime verification.

AgentMiddleware can stop a run before its continuation; FunctionMiddleware or a tool wrapper checks actual results. WorkflowBuilder needs explicit reject and review destinations. Revalidate changed goals, grants, or relevant state; correlate the scoped verdict with traces. Reject malformed input explicitly, route UNKNOWN to review, and stop or escalate on failed runtime postconditions. A postcheck cannot undo a completed side effect, so consequential tools still need pre-call authorization.

Formal foundation. Refutation soundness transfers absence of an abstract goal to absence of a concrete declared run when abstraction preserves steps and goals. Tolerance preserves refutations from a coarser model to a finer one; adding capabilities with unchanged semantics preserves existing witnesses. Subject reduction and session fidelity connect typed sessions to protocol execution under explicit side conditions. Finite-control widened search terminates in the stated fragment, but abstract witnesses may be spurious. Coq mechanizes scoped results, not the complete Python implementation (S9-S10).

Implications / Conclusion

SkillC is an implemented research prototype, not a production service; the MAF adapter is proposed. Its value is evidence before execution, especially when unchanged impossible work would otherwise recur. Practitioners should use deterministic preflight where contracts exist, invoke LLM extraction only when needed, and evaluate completion alongside token savings. The talk equips attendees to separate intent review, static validation, authorization, and runtime verification in their own agent workflows. Our bounded adapters, single-pass live sampling, and limited multi-role coverage do not establish production-wide savings or security. Validate whether the goal is reachable, then verify what execution actually did.

References: related work

- [1] R. Fikes and N. Nilsson. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving. *Artificial Intelligence* 2(3-4), 1971.
- [2] K. Honda, N. Yoshida, and M. Carbone. Multiparty Asynchronous Session Types. *Journal of the ACM* 63(1), 2016.
- [3] L. de Moura and N. Bjørner. Z3: An Efficient SMT Solver. *TACAS, LNCS 4963*, pp. 337-340. Springer, 2008.
- [4] F. Barbanera, M. Dezani-Ciancaglini, and U. de'Liguoro. Un-projectable Global Types for Multiparty Sessions. *PPDP*, pp. 15:1-15:13. ACM, 2024.
- [5] F. Dagnino, P. Giannini, and M. Dezani-Ciancaglini. Deconfined Global Types for Asynchronous Sessions. *Logical Methods in Computer Science* 19(1), pp. 1-41, 2023.
- [6] S. Hu, C. Lu, and J. Clune. Automated Design of Agentic Systems. *ICLR*, 2025.
- [7] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, et al. AFlow: Automating Agentic Workflow Generation. *ICLR*, 2025.
- [8] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica. Why Do Multi-Agent LLM Systems Fail? *arXiv:2503.13657*, 2025.
- [9] L. Miculicich, M. Parmar, H. Palangi, K. D. Dvijotham, M. Montanari, T. Pfister, and L. T. Le. VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation. *arXiv:2510.05156*, 2025.
- [10] N. Paz, K. Pradeep, N. Raghavan, A. Nikirk, and M. Gupta. SkillSpector: A Pre-Publication Security Control for Agent Skills. *Agent Skills Workshop at CAIS*, 2026.